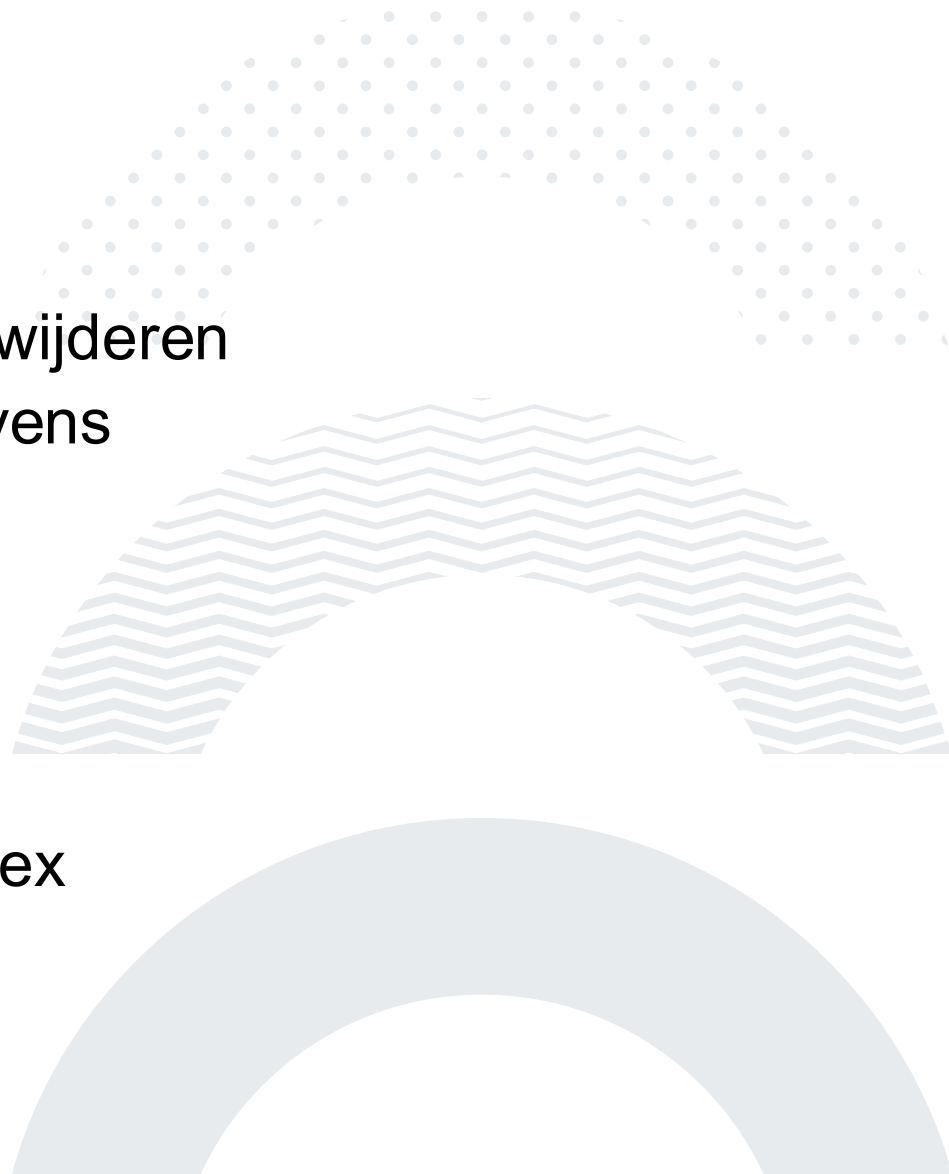


Databases

**Create table/Drop table
insert/update/delete**

Inhoudstafel

- Tabellen creëren
- Tabellen verwijderen of volledige inhoud verwijderen
- Invoeren, wijzigen en verwijderen van gegevens
 - Rijen | records toevoegen
 - Rijen | records verwijderen
 - Rijen | records wijzigen
- Achteraf toevoegen van ...
- Toevoegen of verwijderen van een AK of index
- Schema toevoegen of verwijderen



Tabellen creëren

Hoe tabel-, kolomnamen benoemen ?

- Hier zien we hoe je een ERD omzet naar een database
- Tabelnaam:
 - Databasenaam.Schemaanaam.Tabelnaam
 - Voorbeeld:
 - DB_r1234567.Les.Uitvoerder
 - Databasenaam: (optioneel)
 - = DB_r-nummer
 - Schemaanaam:
 - = 'Les', 'Tennis', ... voor onze voorbeelden
 - Standaard 'dbo'
 - Tabelnaam

Schema

- Om samen horende tabellen, views, ... te groeperen creëren we een schema binnen onze database

```
CREATE SCHEMA Les;  
GO
```

- Verwijderen van een schema
 - Eerst alle tabellen verwijderen

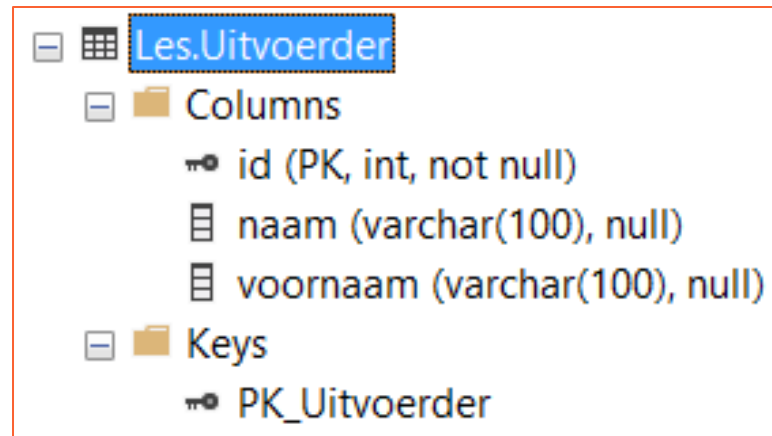
```
DROP SCHEMA Les;  
GO
```

CREATE TABLE

- SQL

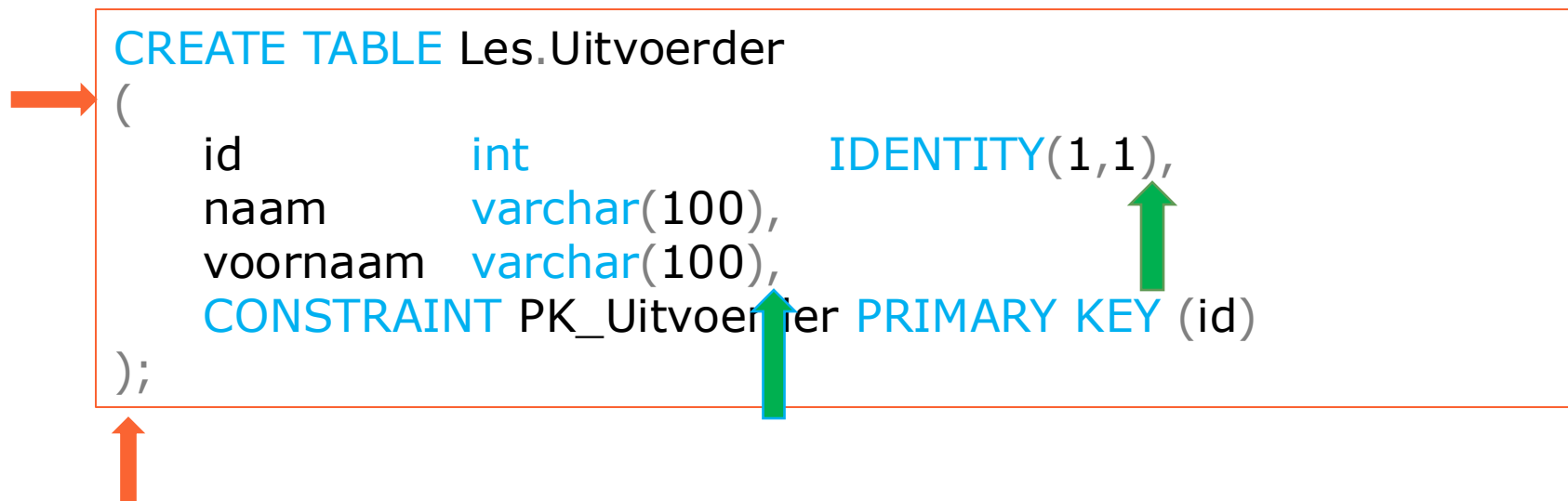
```
CREATE TABLE Les.Uitvoerder  
(  
    id            int            IDENTITY(1,1),  
    naam          varchar(100),  
    voornaam      varchar(100),  
    CONSTRAINT PK_Uitvoerder PRIMARY KEY (id)  
);
```

- Resultaat



CREATE TABLE

- CREATE TABLE maakt tabel *Uitvoerder* aan onder het schema *Les* met de velden *id*, *naam* en *voornaam*.
- De ≠ veldennamen worden tussen de **ronde** haken genoteerd, gescheiden met een **komma**



```
CREATE TABLE Les.Uitvoerder
(
    id          int          IDENTITY(1,1),
    naam        varchar(100),
    voornaam    varchar(100),
    CONSTRAINT PK_Uitvoerder PRIMARY KEY (id)
);
```

The diagram shows the SQL statement for creating the 'Uitvoerder' table in the 'Les' schema. Annotations include: an orange arrow pointing to the opening parenthesis '(', an orange arrow pointing to the closing parenthesis and semicolon ');', a green arrow pointing to the 'ID' column definition, and another green arrow pointing to the 'PRIMARY KEY' constraint.

CREATE TABLE

- De instructie CREATE TABLE wordt in hoofdletters geschreven
- De instructie eindigt met puntkomma ;

```
CREATE TABLE Les.Uitvoerder
(  
    id          int          IDENTITY(1,1),  
    naam        varchar(100),  
    voornaam    varchar(100),  
    CONSTRAINT PK_Uitvoerder PRIMARY KEY (id)  
);
```



CREATE TABLE (velden aanmaken)

veldnaam datatype [CONSTRAINTS]

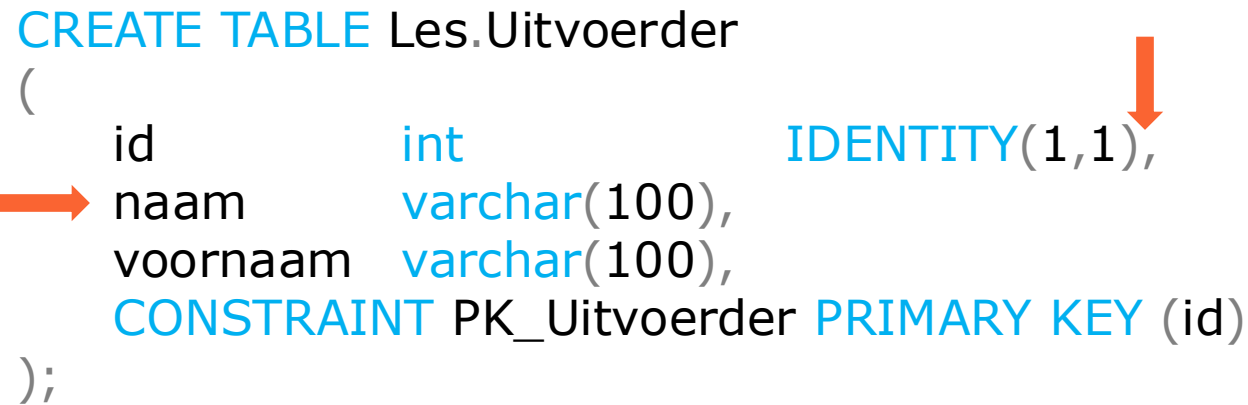
- *veldnaam* = kolomnaam
 - zelf te kiezen
- datatype: kiezen uit de lijst (zie volgende dia's); is afhankelijk van dbms
- constraints: optioneel

```
CREATE TABLE Les.Uitvoerder
(  
→ id          int          IDENTITY(1,1),  
  naam        varchar(100),  
  voornaam    varchar(100),  
  CONSTRAINT PK_Uitvoerder PRIMARY KEY (id)  
);
```

CREATE TABLE (velden aanmaken)

- Achter elke definitie van een veld schrijf je een komma (behalve achter de laatste)
- Per veld gebruik je een andere regel en spring je in

```
CREATE TABLE Les.Uitvoerder
(
    id            int            IDENTITY(1,1),
    naam          varchar(100),
    voornaam      varchar(100),
    CONSTRAINT PK_Uitvoerder PRIMARY KEY (id)
);
```

A diagram illustrating SQL syntax for creating a table. The code is enclosed in a red rectangular box. An orange arrow points from the left to the comma after 'varchar(100)' on the 'naam' line. Another orange arrow points from the top to the comma after 'IDENTITY(1,1)' on the 'id' line. The code is as follows:
`CREATE TABLE Les.Uitvoerder`
`(`
 `id int IDENTITY(1,1),`
 `naam varchar(100),`
 `voornaam varchar(100),`
 `CONSTRAINT PK_Uitvoerder PRIMARY KEY (id)`
`);`

Datatype (numeriek)

`numeric[(m[,d])]`, `decimal [(m[,d])]`

- Voor getallen met vast aantal cijfers na de komma
 - `m` = max. aantal cijfers voor en na de komma samen (standaard = 18)
 - `d` = max. aantal cijfers na de komma (standaard = 0)
- Gebruik in bytes:
 - 5 – 17 bytes

Datatype (numeriek)

- `numeric[(m[,d])]`, `decimal [(m[,d])]`

Voorbeeld:

- `getal` `decimal(5,2)` ➔ -999,00 t/m 999,99
- `waarde` `numeric(5)` ➔ -99999 t/m 99999
- `getal2` `numeric` ➔ positief of negatief getal met max. 18 cijfers

Datatype (numeriek)

tinyint, smallint, int, bigint

- Voor gehele getallen
- Gebruik in bytes:
 - tinyint -> 1
 - smallint -> 2
 - int -> 4
 - bigint -> 8
- Primaire sleutel en refererende keys **altijd** int

Datatype (numeriek)

bit

- Voor geheel getal dat 0, 1 of NULL kan zijn

float(n), real

- Voor getallen met drijvende komma
- n = geeft de precisie van het getal weer
 - n < 25 wordt float toch real
- Gebruik in bytes:
 - real -> 4
 - float -> 8

Datatype (numeriek)

smallmoney, money

- Voor getallen die geld- of valutawaarden bevatten
- Gebruik in bytes:
 - smallmoney -> 4
 - money -> 8

Datatype (datum/tijd)

date

- Bewaart alleen een datum
- Formaat: JJJJ-MM-DD
- '0001-01-01' tot '9999-12-31'
- Default: '1900-01-01'

time

- Bewaart alleen een tijd met een nauwkeurigheid van 100 nanoseconden
- Formaat: hh:mm:ss:nnn

datetime, datetime2, smalldatetime, datetimeoffset

- Bewaart datum en tijd

Datatype (datum/tijd): voorbeeld

Gebruikte datum 23/10/2018

Tijd: 15:45:47 1533333

datum1	tijd1
2018-10-23	15:45:47.1533333

datum2	datum3
2018-10-23 15:45:47.153	2018-10-23 15:45:47.1533333

datum4	datum5
2018-10-23 15:46:00	2018-10-23 15:45:47.1533333 +00:00

```
CREATE TABLE [Les].[DatumTijd]
(
    [id]          int IDENTITY(1,1),
    [datum1]      date,
    [tijd1]       time,
    [datum2]      datetime,
    [datum3]      datetime2,
    [datum4]      smalldatetime,
    [datum5]      datetimeoffset
);
```

Datatype (tekst)

char(n)

- string van *exact* deze lengte (max. 8000)
- n = exacte lengte
- Gebruik in bytes:
 - gedefinieerde lengte

varchar(n)

- string met *variabele* lengte
- n = tekenreekslengte (tussen 1 en 8000)
- Gebruik in bytes:
 - werkelijke grootte + 2 bytes

Datatype (tekst)

text

- Variabele met karakterstring - 2 GB tekstdata

Zie ook SQL Server Data Types

https://www.w3schools.com/sql/sql_datatypes.asp

Constraints

- Constraints zijn beperkingen/voorwaarden op een databank
- Bijvoorbeeld:
 - sommige velden moeten wel of niet ingevuld zijn
 - sommige velden moeten uniek zijn
 - sommige velden vormen de primaire sleutel of refererende sleutel
 - sommige velden mogen alleen bepaalde waarde bevatten
 - sommige velden hebben een standaardwaarde indien ze niet worden ingevuld

Constraints: NULL | NOT NULL

Sommige velden moeten **wel of niet** ingevuld zijn

```
CREATE TABLE Les.Persoon
(  
    id                int                NOT NULL,  
    familienaam       varchar(100) NOT NULL,  
    voornaam          varchar(100) NOT NULL,  
    nicknaam          varchar(100) NULL  
);
```

- *id*, *familienaam* en *voornaam* moeten ingevuld worden (NOT NULL = NNA)
- *nicknaam* mag ingevuld worden (NULL = NA)

Constraints: NULL | NOT NULL

Sommige velden moeten **wel of niet** ingevuld zijn

SERVER-2016.O...8201 - Les.Persoon SQLQuery10.sql - st...(sql_u0068201 (52))*

id	familienaam	voornaam	nicknaam
15	NULL	Marjan	NULL
NULL	NULL	NULL	NULL

Microsoft SQL Server Management Studio



No row was updated.

The data in row 1 was not committed.

Error Source: .Net SqlClient Data Provider.

Error Message: Cannot insert the value NULL into column 'familienaam', table 'Oracle_u0068201.Les.Persoon'; column does not allow nulls. INSERT fails.

The statement has been terminated.

Correct the errors and retry or press ESC to cancel the change(s).

OK

Help

Error Message: Cannot insert the value NULL into column 'familienaam', table 'Oracle_u0068201.Les.Persoon'; column does not allow nulls. INSERT fails.

	id	familienaam	voornaam	nicknaam
	15	Peeters	Marjan	NULL
►*	NULL	NULL	NULL	NULL

Constraints: UNIQUE

Sommige velden moeten **uniek** zijn

```
CREATE TABLE Les.Medewerker
(
    id          int          NOT NULL,
    familienaam varchar(100) NOT NULL,
    voornaam    varchar(100) NOT NULL,
    nicknaam    varchar(100) NULL,
    CONSTRAINT UK_Medewerker_id UNIQUE (id)
);
```

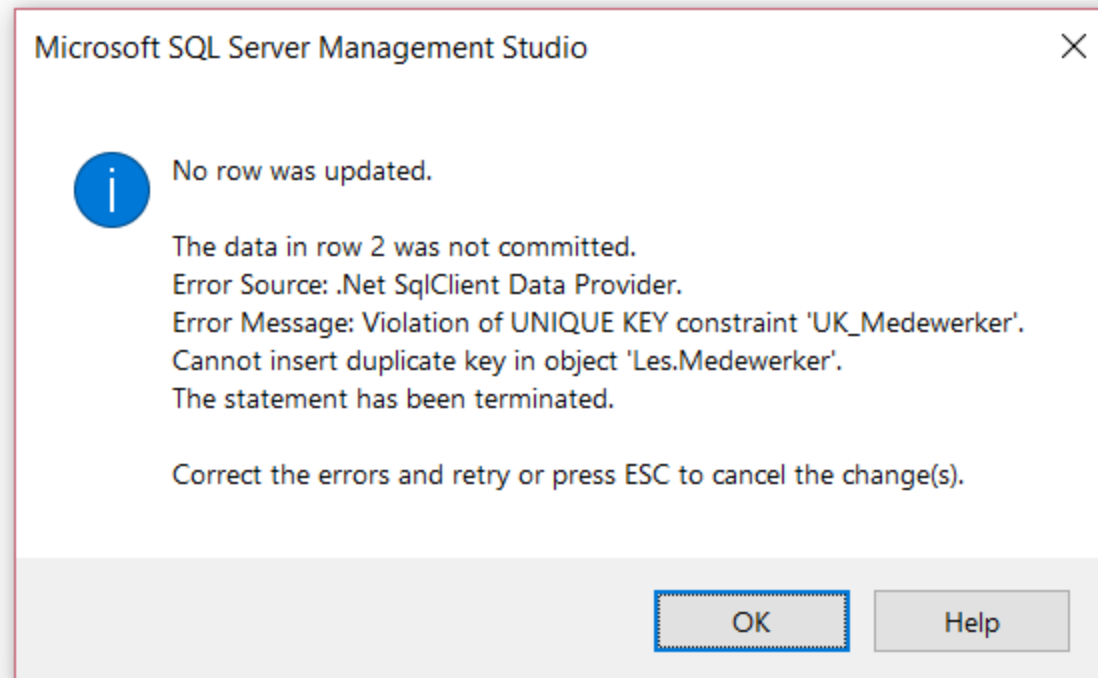


- *id* moet ingevuld worden EN uniek zijn
- *familienaam* mag dubbel voorkomen

Constraints: UNIQUE (id)

- Voorbeeld:

	id	familienaam	voornaam	nicknaam
	100	Maes	Karin	Kaatje
✎	100	Put	Petra	Petje
*	NULL	NULL	NULL	NULL



Error Message: Violation of UNIQUE KEY constraint 'UK_Medewerker'.
Cannot insert duplicate key in object 'Les.Medewerker'.

Constraints: UNIQUE

Sommige velden moeten samen **uniek** zijn

```
CREATE TABLE Les.Medewerker
(
    id          int          NOT NULL,
    familienaam varchar(100) NOT NULL,
    voornaam    varchar(100) NOT NULL,
    nicknaam    varchar(100) NULL,
    CONSTRAINT UK_Medewerker_id_familienaam
               UNIQUE (id,familienaam)
);
```

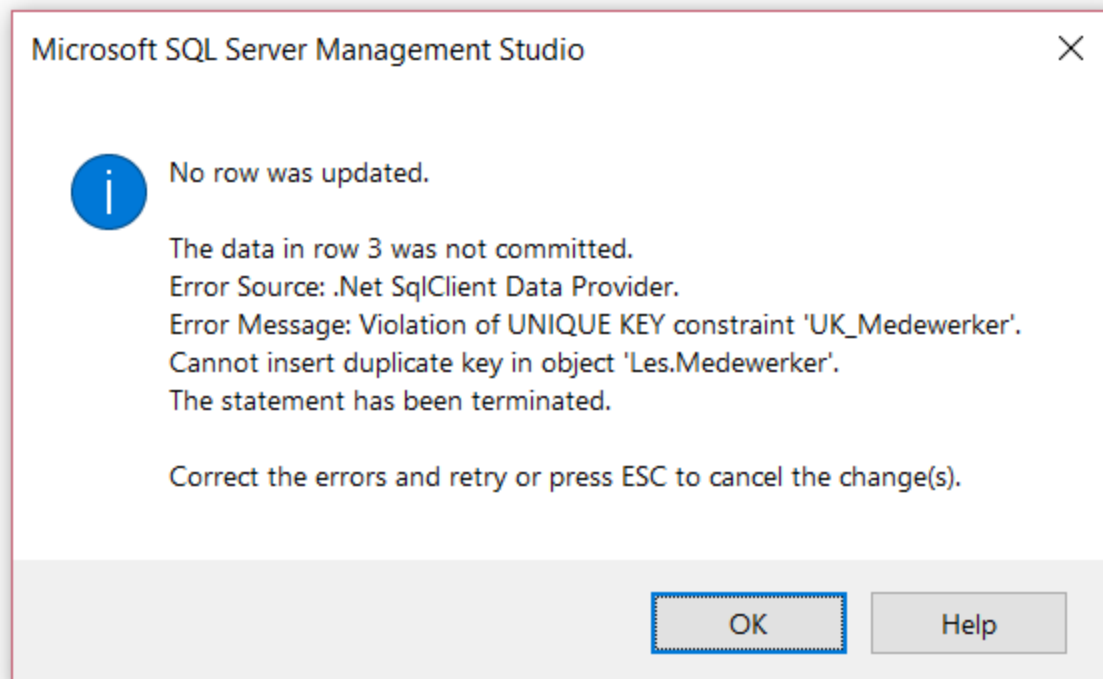


- *id* en *familienaam* moeten ingevuld worden EN samen uniek zijn
- Constraintnaam = **UK**_Tabelnaam_kolomnaam

Constraints: UNIQUE (id, familienaam)

- Voorbeeld:

	id	familienaam	voornaam	nicknaam
	100	Maes	Karin	Kaatje
	100	Peeters	Frank	Frankse
...	100	Maes	Louis	Loe
*	NULL	NULL	NULL	NULL



Constraints: PRIMARY KEY

- Sommige velden vormen de **primaire sleutel** of **primary key**

```
CREATE TABLE Les.Persoon
(
    id                int,
    familienaam       varchar(100) NOT NULL,
    voornaam          varchar(100) NOT NULL,
    nicknaam          varchar(100) NULL,
    CONSTRAINT PK_Persoon PRIMARY KEY (id) ←
```

- *id* moet ingevuld worden en is de primaire sleutel van tabel *Persoon*

Constraints: PRIMARY KEY

`CONSTRAINT` naamConstraint `PRIMARY KEY` (veldnaam(en))

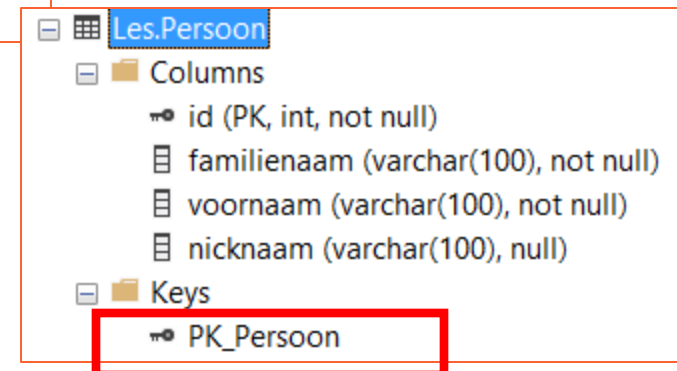
- naamConstraint = `PK_Tabelnaam`
- typeConstraint = `PRIMARY KEY`
- Veldnaam = meestal id

NOT NULL is bij PRIMARY KEY niet nodig, wordt automatisch toegevoegd

Constraints: PRIMARY KEY

CONSTRAINT naamConstraint **PRIMARY KEY** (veldnaam(en))

```
CREATE TABLE Les.Persoon
(  
    id                int,  
    familienaam      varchar(100) NOT NULL,  
    voornaam         varchar(100) NOT NULL,  
    nicknaam         varchar(100) NULL,  
    CONSTRAINT PK_Persoon PRIMARY KEY (id)  
);
```

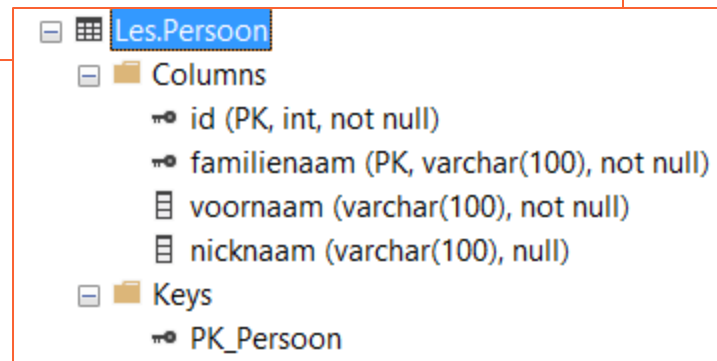


Constraints: PRIMARY KEY

- *id* en *familienaam* moeten ingevuld worden en vormen samen de primaire sleutel van tabel *Persoon*

```
CREATE TABLE Les.Persoon
(
    id                int,
    familienaam       varchar(100) NOT NULL,
    voornaam          varchar(100) NOT NULL,
    nicknaam          varchar(100) NULL,
    CONSTRAINT PK_Persoon PRIMARY KEY (id,familienaam)
);
```

Alleen bij **oude**
database tabellen



Constraints: PRIMARY KEY + autonummering

id wordt automatisch verhoogd

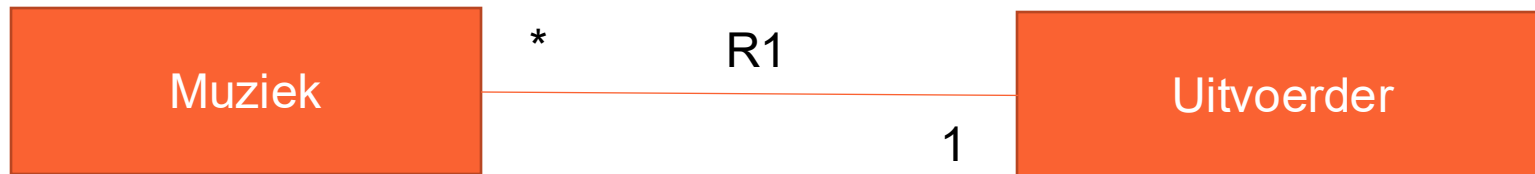
- IDENTITY (beginwaarde, verhoogwaarde)
- PK is normaal met IDENTITY()

```
CREATE TABLE Les.Persoon
(
    id                int                IDENTITY(1,1),
    familienaam       varchar(100)      NOT NULL,
    voornaam          varchar(100)      NOT NULL,
    nicknaam          varchar(100)      NULL,
    CONSTRAINT PK_Persoon PRIMARY KEY (id)
);
```



Constraints: FOREIGN KEY

- Sommige velden vormen de **foreign key** of **refererende sleutel**
- Een referende sleutel of foreign key geeft een relatie tussen 2 tabellen weer



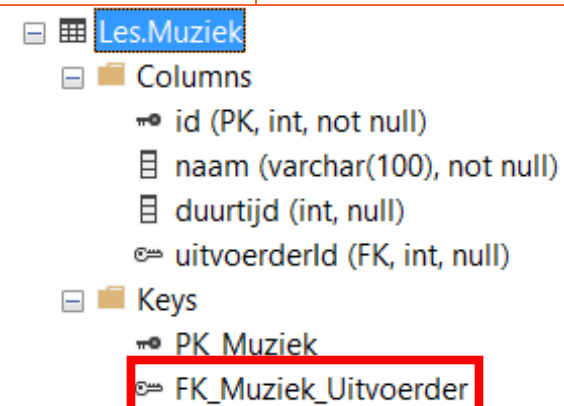
R1: Muziek wordt gemaakt door slechts één uitvoerder; een uitvoerder kan meerdere muziekCd's gemaakt hebben.

Constraints: FOREIGN KEY

- Een referende sleutel of foreign key geeft een relatie tussen 2 tabellen weer

```
CREATE TABLE Les.Muziek
(
    id                int                IDENTITY(1,1),
    naam              varchar(100) NOT NULL,
    duurtijd          int,
    uitvoerderId      int,
    CONSTRAINT PK_Muziek
        PRIMARY KEY (id),
    CONSTRAINT FK_Muziek_Uitvoerder
        FOREIGN KEY (uitvoerderId)
        REFERENCES Les.Uitvoerder(id)
);
```

Een uitvoerderId kan NULL zijn
als hij onbekend is



Constraints: FOREIGN KEY

CONSTRAINT naamConstraint **FOREIGN KEY** (kolomnaam1) **REFERENCES** tabelnaamPK (kolomnaam2)

- naamConstraint = **FK**_Tabelnaam_TabelnaamPK
- typeConstraint = **FOREIGN KEY**
- kolomnaam1 = kolom in deze tabel (= FK)
- kolomnaam2 = kolom uit de PK-tabel (meestal id)

```
CONSTRAINT FK_Muziek_Uitvoerder  
  FOREIGN KEY (uitvoerderId)  
  REFERENCES Les.Uitvoerder(id)
```

Constraints: FOREIGN KEY

CONSTRAINT naamConstraint **FOREIGN KEY** (kolomnaam1) **REFERENCES** tabelnaamPK (kolomnaam2)

- **REFERENCES** geeft weer:
 - met welke tabel er een relatie gemaakt wordt
 - met welk veld van deze tabel (= PK)

```
CONSTRAINT FK_Muziek_Uitvoerder  
  FOREIGN KEY (uitvoerderId)  
  REFERENCES Les.Uitvoerder(id)
```

Constraints: CHECK

- sommige velden mogen **alleen bepaalde waarden** bevatten
 - In het veld *geslacht* mag alleen 'M', 'm', 'V' of 'v' ingevuld worden

```
CREATE TABLE Les.Persoon
(
    id                int      IDENTITY(1,1),
    familienaam       varchar(100) NOT NULL,
    voornaam          varchar(100),
    geslacht           char,
    CONSTRAINT PK_Persoon PRIMARY KEY (id),
    CONSTRAINT CK_Persoon_geslacht
        CHECK (geslacht = 'M' OR geslacht = 'V')
);
```

Constraints: CHECK

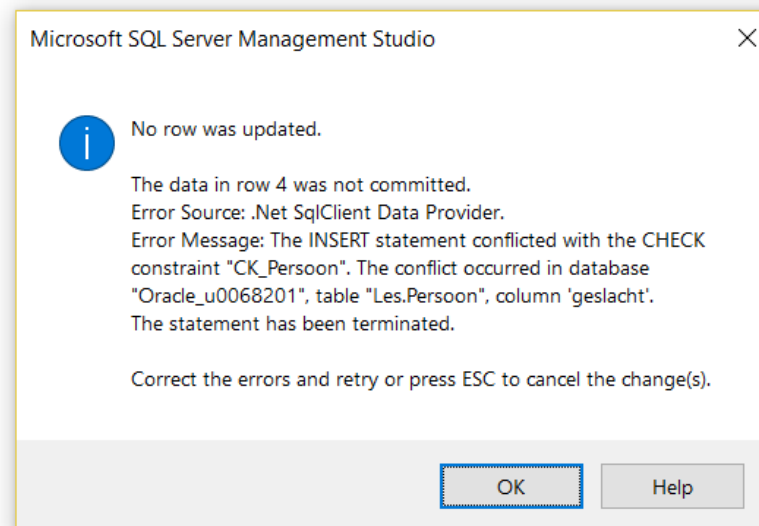
CONSTRAINT naamConstraint **CHECK** (voorwaarde(n))

- naamConstraint = **CK**_Tabelnaam_kolomnaam
- typeConstraint = **CHECK**
- voorwaarde(n):
 - (geslacht = '**M**' OR geslacht = '**V**')
 - of-voorwaarde
 - voldoen aan één van de twee is voldoende
 - (geslacht = '**M**' AND land = '**België**')
 - en-voorwaarde
 - beide voorwaarden moeten voldaan worden

Constraints: CHECK

- In *geslacht* mag men 'M', 'm', 'V' of 'v' invullen.
- Andere waarden worden niet toegelaten

	id	familienaam	voornaam	geslacht	postcode	gemeente	land
	1	Maes	Frank	m	2440	Geel	België
	2	Put	Katrien	v	2440	Geel	België
	3	Claes	Louis	M	2400	Mol	België
	NULL	Jannes	Piet	p	2440	Geel	België
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL



Constraints: DEFAULT

- Sommige velden hebben een **standaardwaarde** indien ze niet worden ingevuld

```
CREATE TABLE Les.Persoon
(
    id            int IDENTITY(1,1),
    familienaam  varchar(100) NOT NULL,
    voornaam     varchar(100),
    geslacht     char,
    postcode     varchar(4),
    gemeente     varchar(100),
    land         varchar(100)
    CONSTRAINT DF_Persoon_land
        DEFAULT 'België',
    CONSTRAINT PK_Persoon PRIMARY KEY (id)
);
```

Geen komma
constraint hoort bij land

Constraints: DEFAULT

`CONSTRAINT` naamConstraint `DEFAULT` (waarde)

- naamConstraint = `DF_Tabelnaam_kolomnaam`
- typeConstraint = `DEFAULT`

Voorbeeld:

```
CONSTRAINT DF_Persoon_land  
    DEFAULT 'België'
```


Waar constraint noteren?

Kolom- of tabelconstraint ?

Kolomconstraint:

- Alleen als het geldig is voor één kolom
- Voorbeeld:

```
id          int          IDENTITY(1,1),
naam        varchar(100) NOT NULL,
voornaam    varchar(100) NULL,
land        varchar(100)
CONSTRAINT DF_Persoon_land
          DEFAULT 'België'
```

- NULL en NOT NULL altijd op kolomniveau

Waar constraint noteren?

Tabelconstraint:

- Altijd mogelijk
 CONSTRAINT naamConstraint
 TYPE CONSTRAINT (betreffendeKolom(men))
- Voorbeelden:

```
CONSTRAINT PK_Muziek PRIMARY KEY (id),  
CONSTRAINT FK_Muziek_Uitvoerder  
    FOREIGN KEY (uitvoerderId)  
    REFERENCES Uitvoerder(id),  
CONSTRAINT UK_Persoon UNIQUE (id, familienaam),  
CONSTRAINT CK_Persoon_geslacht  
    CHECK (geslacht = 'M' OR geslacht = 'V')
```

DTC, DTR, DTN

DTC: ON DELETE CASCADE

- Als je een rij verwijdert waarnaar in een andere tabel verwezen wordt via een FK, dan worden de rijen die de FK bevatten **ook verwijderd**.

DTR: ON DELETE RESTRICT

- Rijen waarnaar verwezen wordt in een andere tabel kunnen **niet** verwijderd worden.

DTN: ON DELETE SET NULL

- Als je een rij verwijdert waarnaar in een andere tabel verwezen wordt via een FK, dan wordt de FK op **NULL gezet**.

DTC: ON DELETE CASCADE

CONSTRAINT FOREIGN KEY uitbreiden met:

- **ON DELETE CASCADE**
- Goed nadenken !!!
- ALLEEN bij associatie tabellen

```
CREATE TABLE Les.Orderlijn
(
    id            int            IDENTITY(1,1),
    orderId       int,
    productId     int,
    hoeveelheid  int            DEFAULT 1,
    CONSTRAINT PK_Orderlijn PRIMARY KEY (id),
    CONSTRAINT FK_Orderlijn_Order
        FOREIGN KEY (orderId)
        REFERENCES Les.Order (id) ON DELETE CASCADE
);
```

Indien je een order verwijderd, worden de bijhorende orderlijnen ook verwijderd.

DTR: ON DELETE RESTRICTED

CONSTRAINT FOREIGN KEY uitbreiden met:

- **niets toevoegen**
- meestal gebruikt

```
CREATE TABLE Les.Orderlijn
(
    id            int            IDENTITY(1,1),
    orderId       int,
    productId     int,
    hoeveelheid  int            DEFAULT 1,
    CONSTRAINT PK_Orderlijn PRIMARY KEY (id),
    CONSTRAINT FK_Orderlijn_Product
        FOREIGN KEY (productId)
        REFERENCES Les.Product (id)
);
```

Je kan geen product verwijderen als er nog orderlijnen bestaan voor dit product.

DTN: ON DELETE SET NULL

CONSTRAINT FOREIGN KEY uitbreiden met:

- **ON DELETE SET NULL**

```
CREATE TABLE Les.Orderlijn
(
    id            int            IDENTITY(1,1),
    orderId       int,
    productId     int,
    hoeveelheid  int            DEFAULT 1,
    CONSTRAINT PK_Orderlijn PRIMARY KEY (id),
    CONSTRAINT FK_Orderlijn_Product
        FOREIGN KEY (productId)
        REFERENCES Les.Product (id) ON DELETE SET NULL
);
```

Als je een product verwijdert, wordt in alle orderlijnen dit product op NULL geplaatst.

Volgorde van aanmaken

Tabellen moeten aangemaakt worden in volgorde van de relaties.

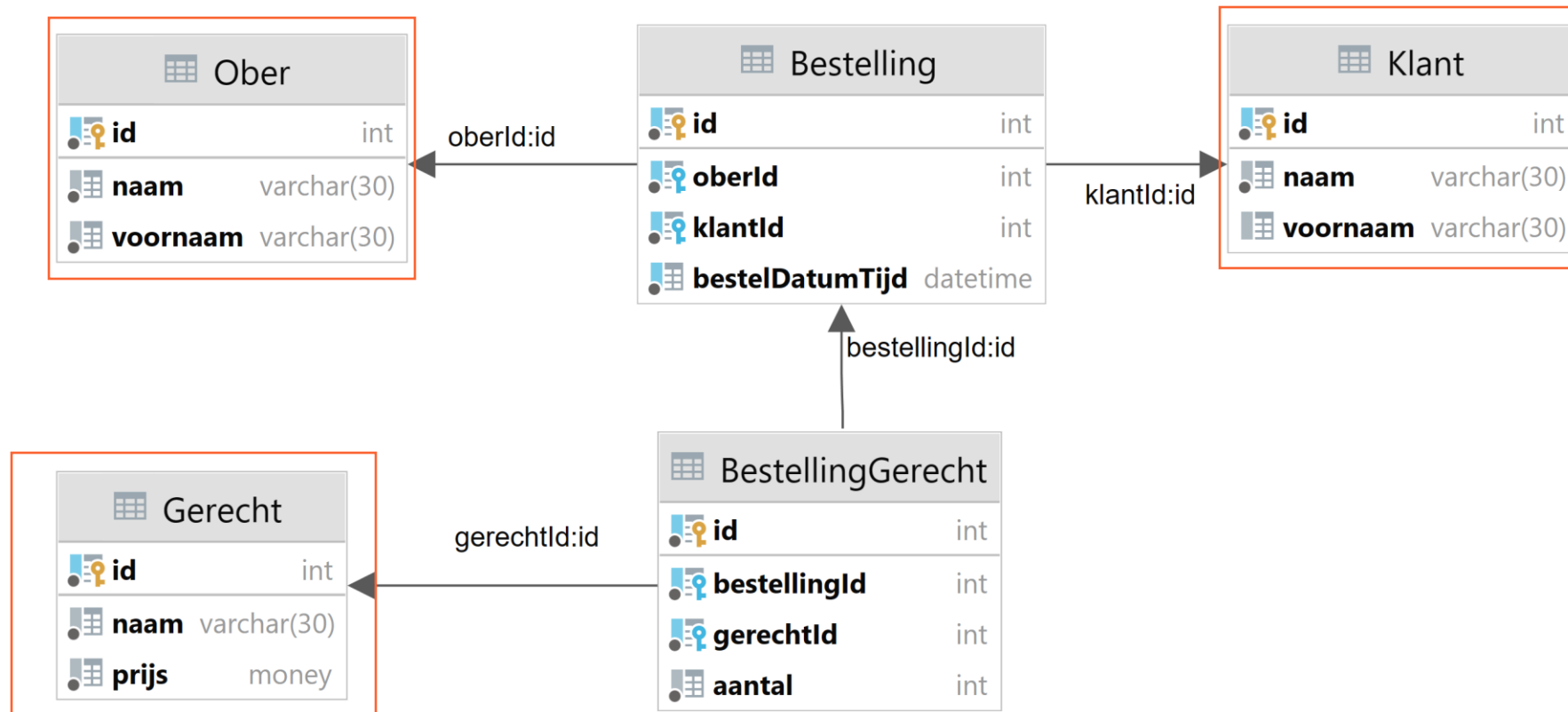
- Eerst de tabellen zonder foreign keys
- Vervolgens de tabellen die een FK bevatten

Voorbeeld:



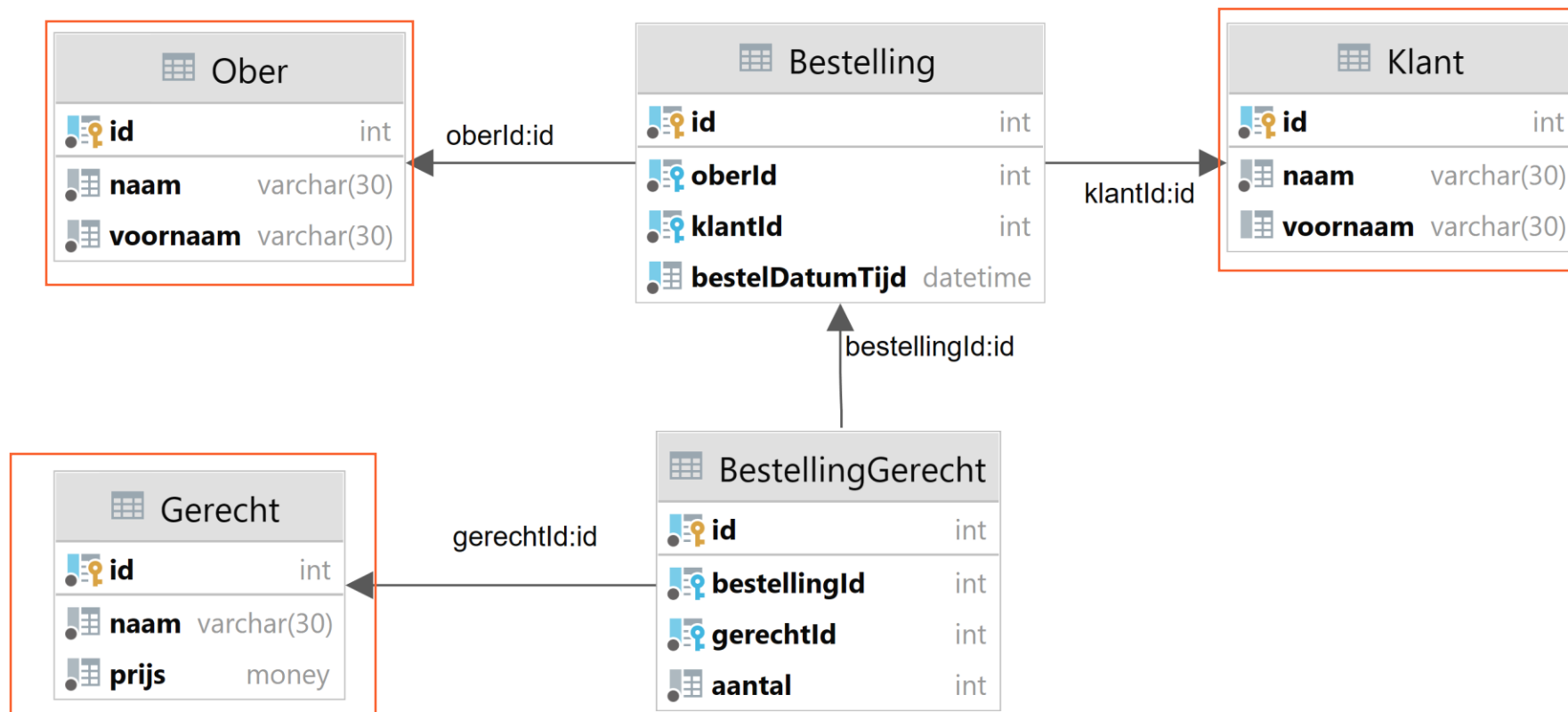
Volgorde van aanmaken

- Tabellen zonder foreign keys



Volgorde van aanmaken

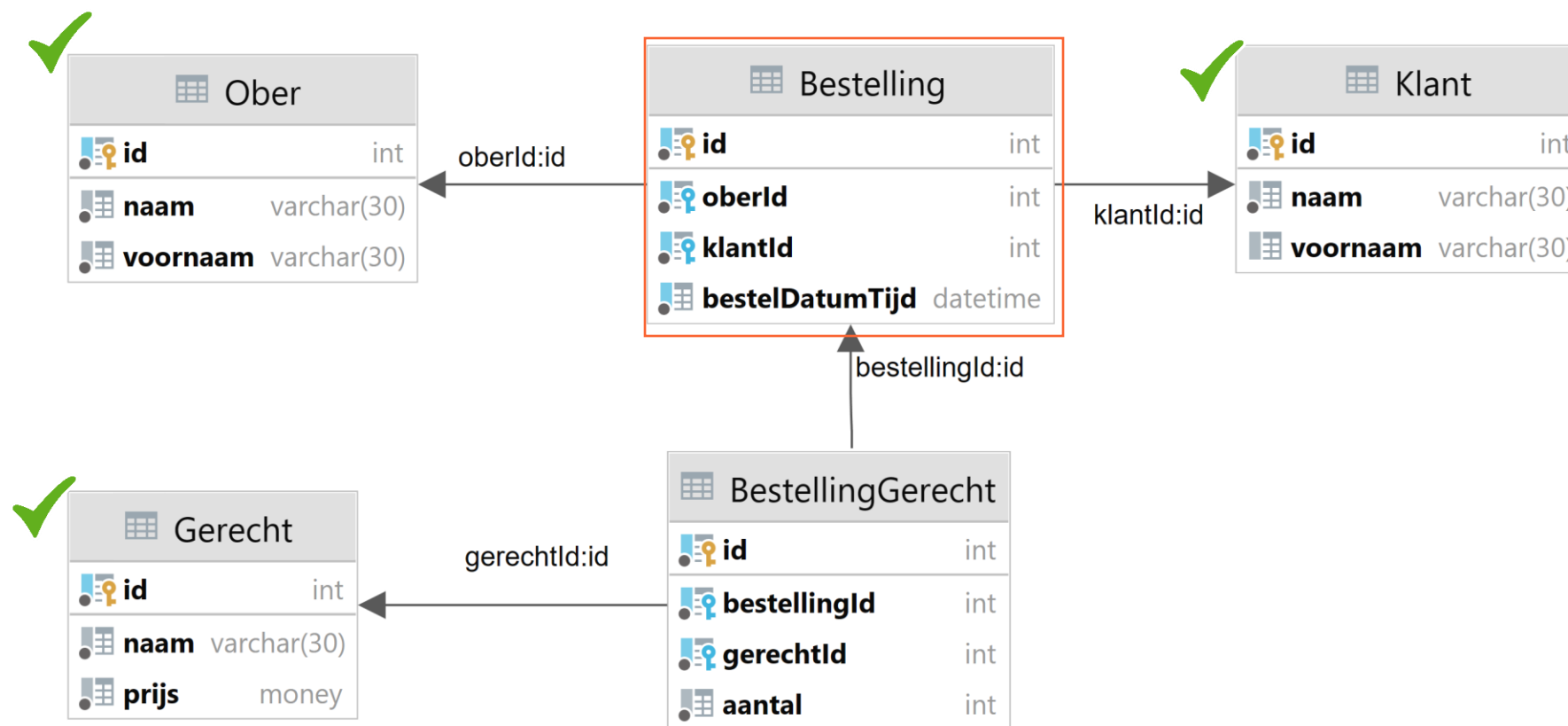
- Tabellen zonder foreign keys



De volgorde waarin ze **onderling** aangemaakt worden is niet belangrijk.

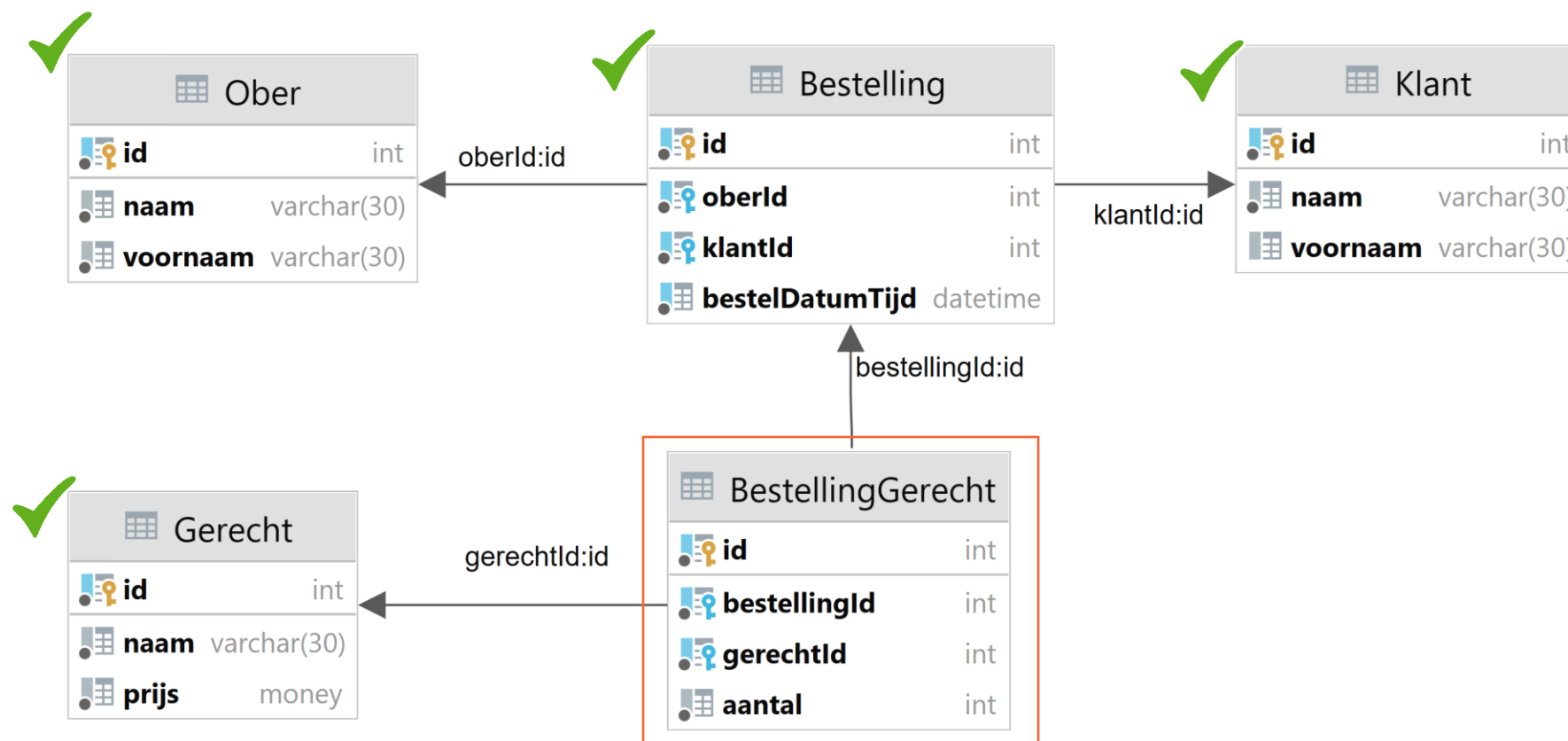
Volgorde van aanmaken

- Tabellen zonder foreign keys
- Tabellen met **uitsluitend** foreign keys naar de reeds gemaakte tabellen



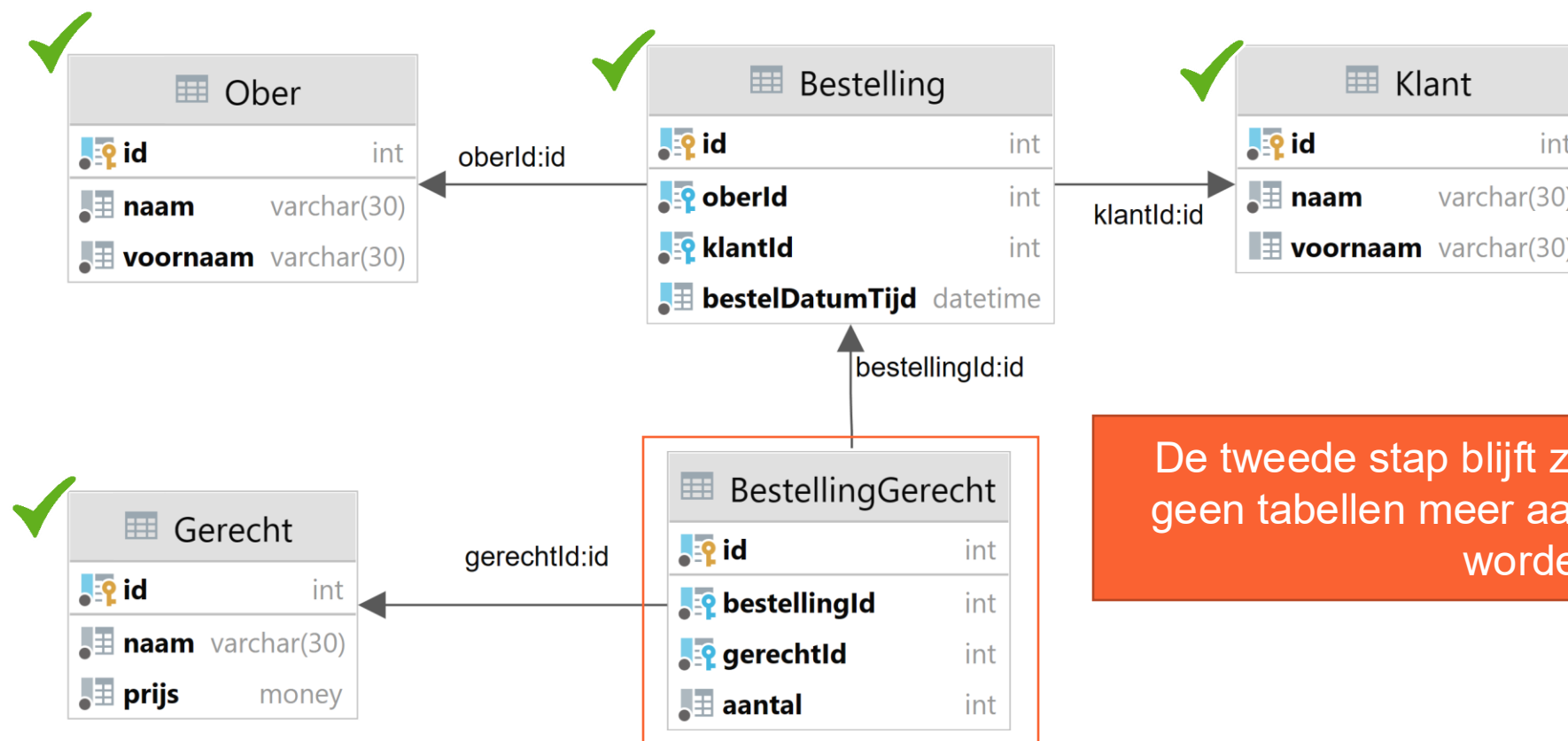
Volgorde van aanmaken

- Tabellen zonder foreign keys
- Tabellen met **uitsluitend** foreign keys naar de reeds gemaakte tabellen ↻



Volgorde van aanmaken

- Tabellen zonder foreign keys
- Tabellen met **uitsluitend** foreign keys naar de reeds gemaakte tabellen



De tweede stap blijft zich herhalen tot er geen tabellen meer aangemaakt moeten worden

Tabellen verwijderen Rijen verwijderen

Tabel verwijderen

```
DROP TABLE Les.Muziek;
```

DROP TABLE werkt **niet** als er nog andere tabellen naar deze tabel verwijzen (FK)

- Tenzij CASCADE CONSTRAINT aanwezig is

Inhoud van tabel verwijderen

```
TRUNCATE TABLE Les.Uitvoerder;
```

- De tabel zelf wordt niet verwijderd
- Alleen de bestaande records worden verwijderd
- *id* start terug vanaf de startwaarde (IDENTITY)

- Voor

WIN-10BQPGS89DM.O...- Les.Uitvoerder			
	id	naam	voornaam
	1	Wouters	Koen
	2	Tura	Will
	3	Jackson	Michaël
▶*	NULL	NULL	NULL

- Na

WIN-10BQPGS89DM.O...- Les.Uitvoerder			
	id	naam	voornaam
▶*	NULL	NULL	NULL

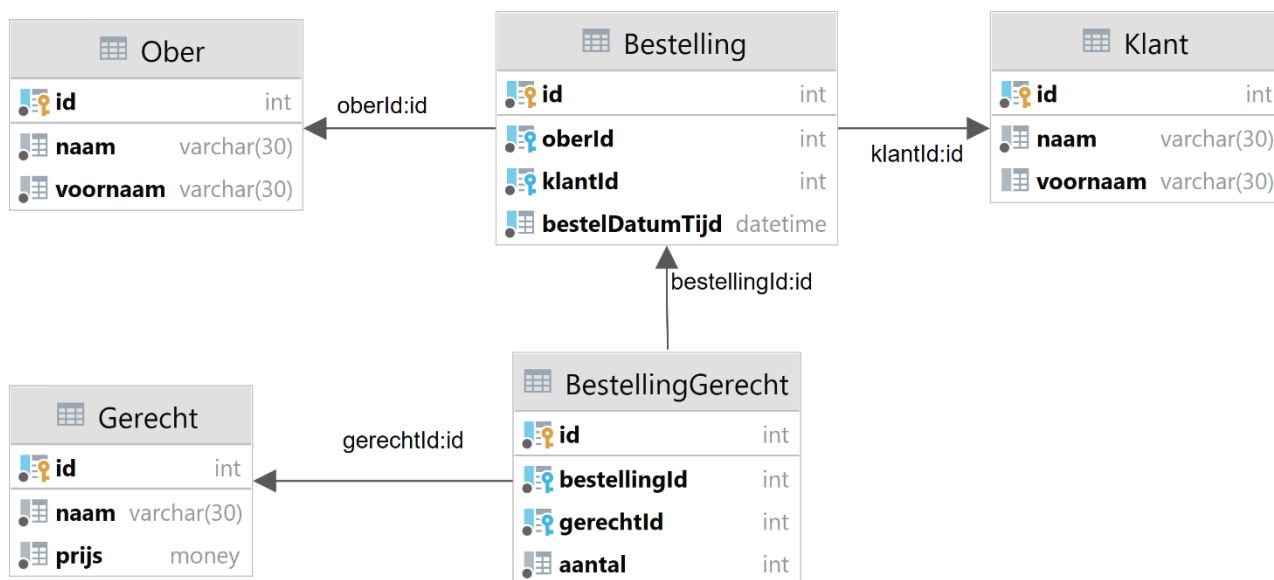
WIN-10BQPGS89DM.O...- Les.Uitvoerder			
	id	naam	voornaam
	1	Wouters	Koen
▶*	NULL	NULL	NULL

Volgorde van verwijderen

Tabellen moeten verwijderd worden in volgorde van de relaties.

- Eerst de tabellen **met** foreign keys
- Vervolgens de tabellen die geen FK bevatten

Voorbeeld:



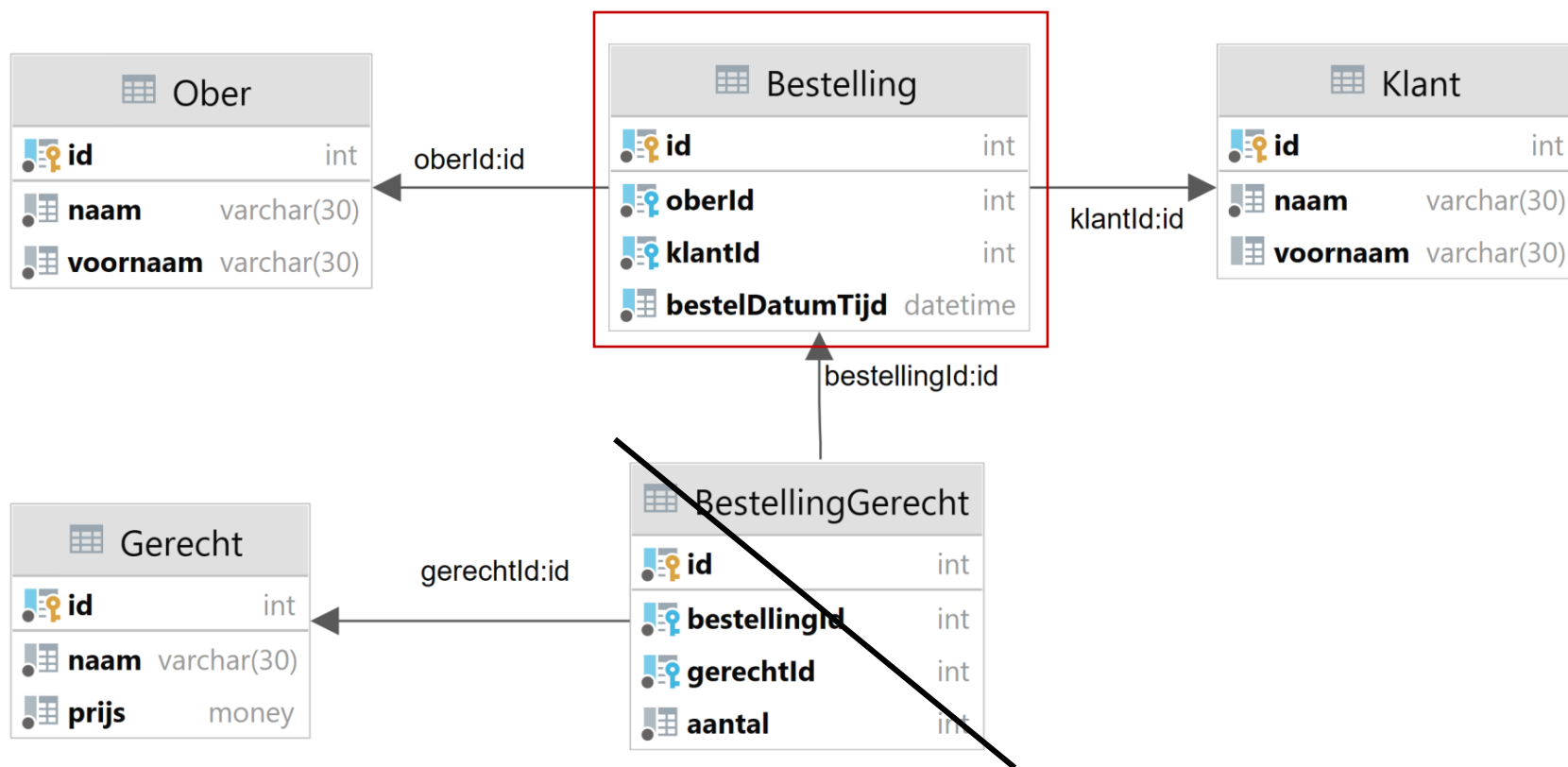
Volgorde van verwijderen

- Tabellen met foreign keys **waarnaar niet verwezen wordt in andere tabellen**



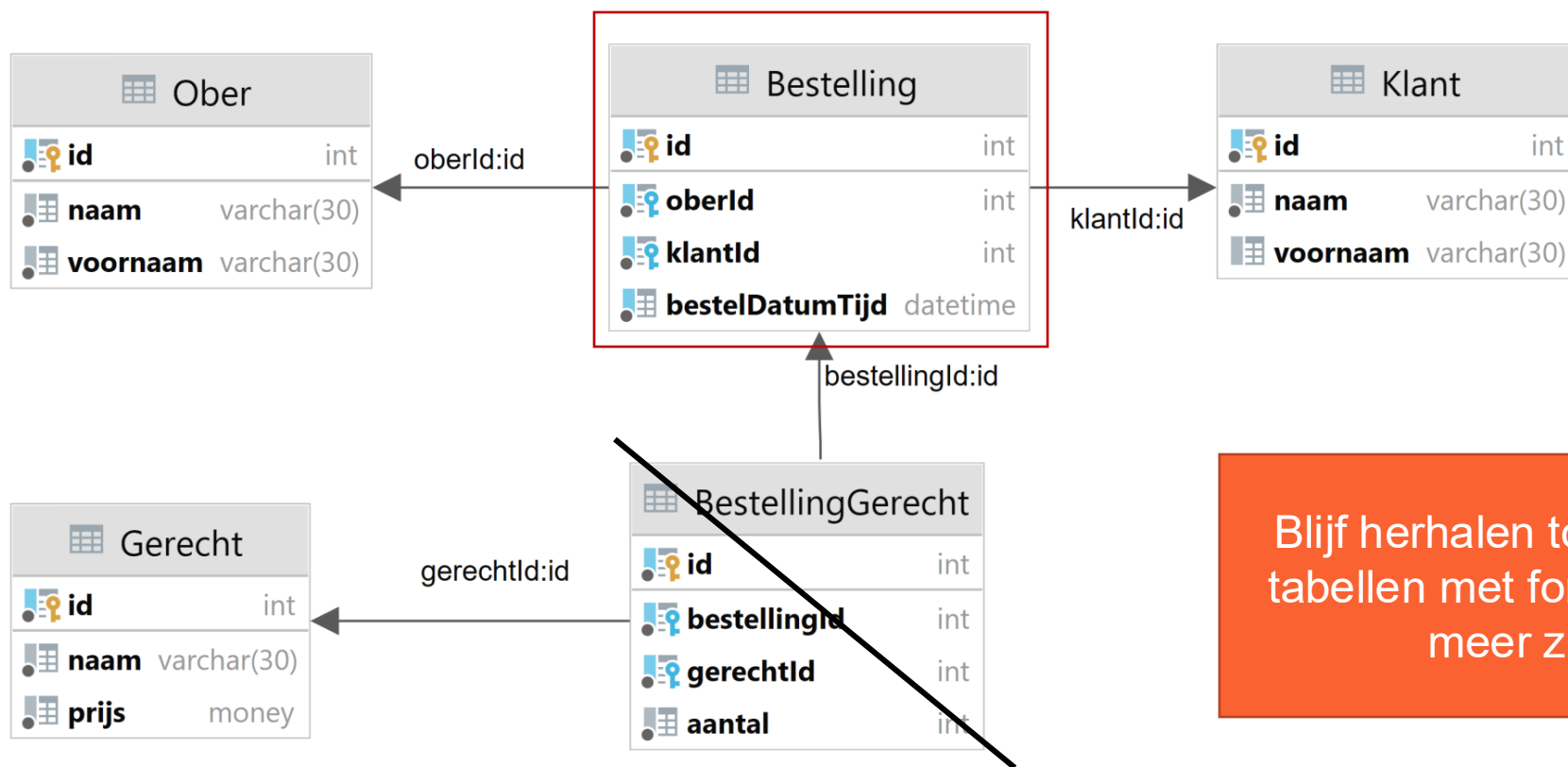
Volgorde van verwijderen

- Tabellen met foreign keys **waarnaar niet verwezen wordt in andere tabellen**



Volgorde van verwijderen

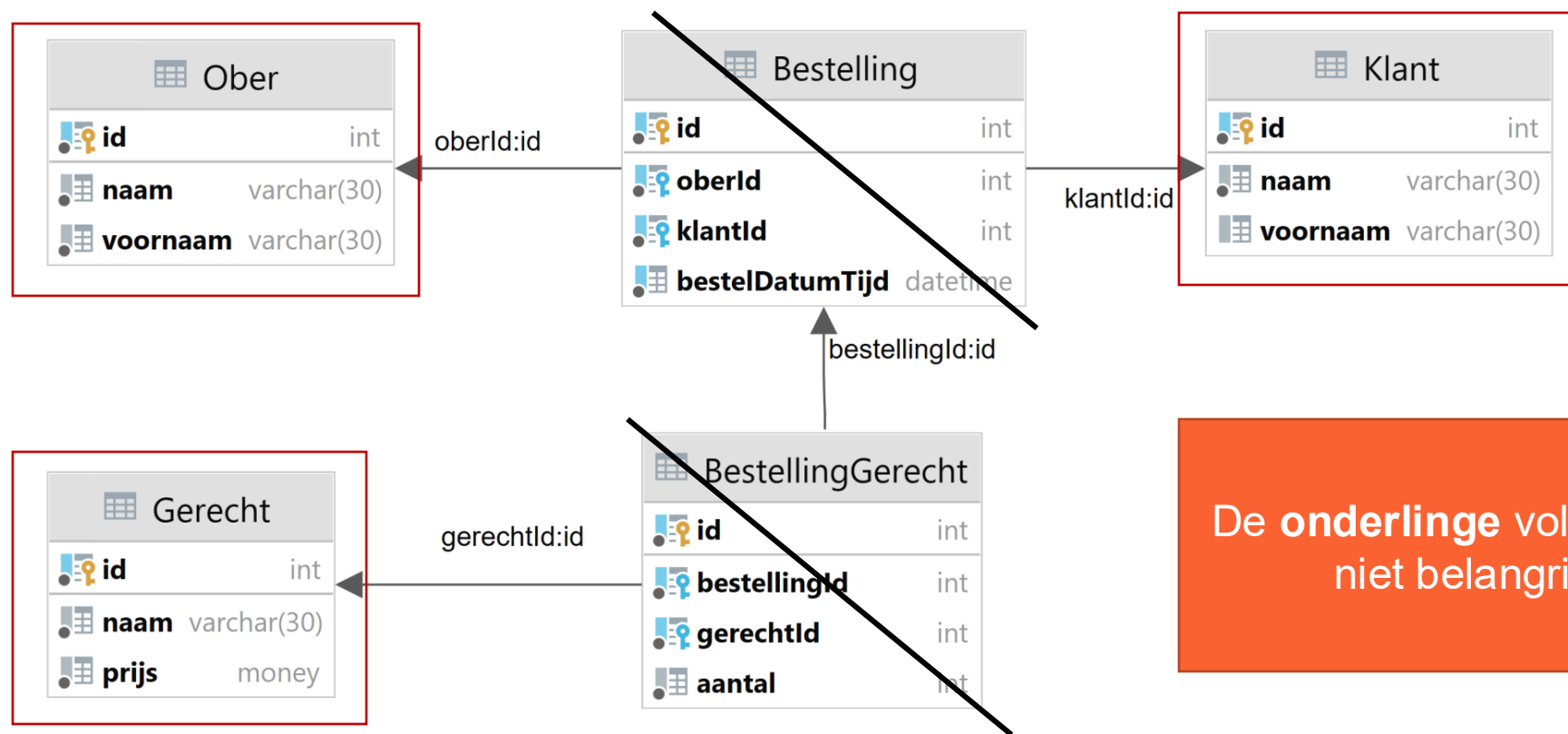
- Tabellen met foreign keys **waarnaar niet verwezen wordt in andere tabellen**



Blijf herhalen tot er geen tabellen met foreign keys meer zijn

Volgorde van verwijderen

- Tabellen met foreign keys **waarnaar niet verwezen wordt in andere tabellen**
- Alle tabellen zonder foreign keys



De **onderlinge** volgorde is niet belangrijk

Invoeren, wijzigen en verwijderen van gegevens

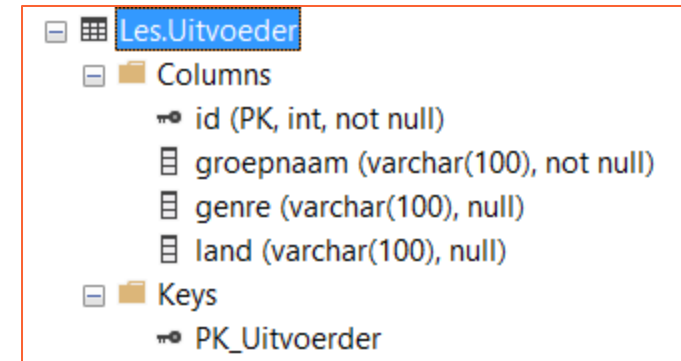
Rijen toevoegen (manier 1)

```
INSERT INTO Schemanaam.Tabelnaam  
  (kolomnaam1, kolomnaam2, ...)  
  VALUES (waarde1, waarde2, ...);
```

- Aantal waarden moet overeenkomen met het aantal kolommen in de tabel
- Type moet overeenkomen
- Kolomvolgorde moet gerespecteerd worden
- Niet vermelde kolommen krijgen de NULL-waarde (= waarde onbekend)
- NULL kan ook als waarde worden meegegeven

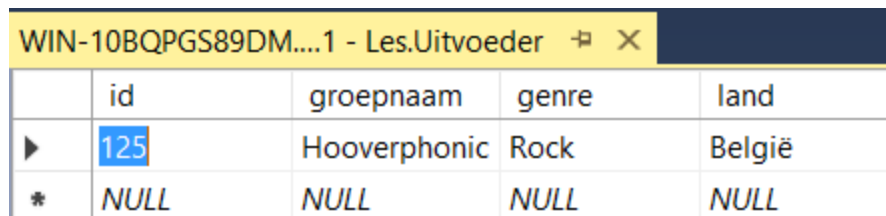
Rijen toevoegen: voorbeeld

Voeg de groep 'Hooverphonic'
toe aan de tabel Uitvoerder'
(id=125, genre='Rock', land='België')



Les.Uitvoeder
Columns
id (PK, int, not null)
groepnaam (varchar(100), not null)
genre (varchar(100), null)
land (varchar(100), null)
Keys
PK_Uitvoeder

```
INSERT INTO Les.Uitvoeder  
  (id, groepnaam, genre, land)  
VALUES (125, 'Hooverphonic', 'Rock', 'België');
```



	id	groepnaam	genre	land
▶	125	Hooverphonic	Rock	België
*	NULL	NULL	NULL	NULL

Rijen toevoegen: voorbeeld

```
INSERT INTO Les.Uitvoeder  
  (id, groepnaam, genre, land)  
VALUES (126, 'Clouseau', 'Pop', NULL);
```

```
INSERT INTO Les.Uitvoeder  
  (id, groepnaam, genre)  
VALUES (127, 'Police', 'Rock');
```

WIN-10BQPGS89DM....1 - Les.Uitvoeder ✕ dia 56.sql - studer				
	id	groepnaam	genre	land
▶	125	Hooverphonic	Rock	België
	126	Clouseau	Pop	NULL
	127	Police	Rock	NULL
*	NULL	NULL	NULL	NULL

Rijen toevoegen (manier 2)

- Aantal waarden moet overeenkomen met het aantal kolommen in de tabel
- Type moet overeenkomen
- Kolomvolgorde moet gerespecteerd worden
- Niet vermelde kolommen krijgen de NULL-waarde (= waarde onbekend)
- NULL kan ook als waarde worden meegegeven

Rijen toevoegen (manier 2)

```
INSERT INTO Schemanaam.Tabelnaam  
  (kolomnaam1, kolomnaam2, ...)  
  SELECT-commando;
```

SELECT commando komt in Les 3 aan bod!

- SELECT-commando levert 0, 1 of meer rijen op die worden toegevoegd.
- Het resultaat van het SELECT-commando moet wat betreft aantal en type van de kolommen overeenkomen met het gespecificeerde.
- Kolomnamen moeten worden vermeld wanneer men afwijkt van de volgorde of het aantal in de tabel.
- In het SELECT-commando mag geen ORDER BY-component voorkomen.

Rijen toevoegen: voorbeeld

- Stel dat een (lege) tabel *BelgischeUitvoerder* dezelfde structuur heeft als de tabel *Uitvoerder*.
Vul deze tabel met de gegevens van de Belgische uitvoerders.

```
INSERT INTO Les.BelgischeUitvoeder  
  SELECT *  
  FROM   Les.Uitvoeder  
  WHERE  land = 'België';
```

- Let op: SELECT wordt ingesprongen alsook FROM en WHERE

Rijen toevoegen: voorbeeld

Voor:

- BelgischeUitvoerder
- Uitvoerder

WIN-10BQPGS89DM....lgischeUitvoerder ✕ create uitvoerde				
	id	groepnaam	genre	land
*	NULL	NULL	NULL	NULL

WIN-10BQPGS89DM....1 - Les.Uitvoeder ✕ SQLQuery1.sql - s				
	id	groepnaam	genre	land
	125	Hooverpho...	Rock	België
	126	Clouseau	Pop	NULL
	127	Police	Rock	NULL
	128	De Kreuners	Rock	België
	129	K3	Pop	België
▶*	NULL	NULL	NULL	NULL

Na:

WIN-10BQPGS89DM....lgischeUitvoeder ✕ SQLQuery1.sql - s				
	id	groepnaam	genre	land
▶	125	Hooverpho...	Rock	België
	128	De Kreuners	Rock	België
	129	K3	Pop	België
*	NULL	NULL	NULL	NULL

Rijen verwijderen

```
DELETE FROM Schemanaam.Tabelnaam  
WHERE voorwaarde;
```

- Indien er geen WHERE component voorkomt, wordt de hele tabel “leeg” gemaakt.
- Opgelet wanneer er relaties bestaan tussen tabellen!

Rijen verwijderen: voorbeeld

- Verwijder alle uitvoerders van genre 'rock'

```
DELETE FROM Les.Uitvoeder  
WHERE genre = 'rock';
```

- Voor:

WIN-10BQPGS89DM....1 - Les.Uitvoeder				
SQLQuery1.sql - s				
	id	groepnaam	genre	land
	125	Hooverpho...	Rock	België
	126	Clouseau	Pop	NULL
	127	Police	Rock	NULL
	128	De Kreuners	Rock	België
	129	K3	Pop	België
►*	NULL	NULL	NULL	NULL

Na:

WIN-10BQPGS89DM....1 - Les.Uitvoeder				
dia 60.sql - studer				
	id	groepnaam	genre	land
►	126	Clouseau	Pop	NULL
	129	K3	Pop	België
*	NULL	NULL	NULL	NULL

Rijen wijzigen

- Elke rij die voldoet aan de voorwaarde in de WHERE-component wordt gewijzigd.

```
UPDATE Schemanaam.Tabelnaam  
SET kolomnaam1 = uitdrukking,  
    kolomnaam2 = uitdrukking,  
    kolomnaam3 = uitdrukking,  
...  
WHERE voorwaarde;
```

Rijen wijzigen: voorbeeld

- Verander het genre 'rock' in 'rockabilly' voor Belgische groepen.

```
UPDATE Les.Uitvoeder  
SET genre = 'Rockabilly'  
WHERE genre = 'rock'  
AND land = 'België';
```

- Voor:

	id	groepnaam	genre	land
	126	Clouseau	Pop	NULL
	129	K3	Pop	België
	130	The Bangle	Rock	USA
	131	The Black K...	Rock	USA
	132	Balthazar	Rock	België
	133	De Kreuners	Rock	België
	134	The Scabs	Rock	België
	135	Das Pop	Britpop	België
	136	Soulsister	Pop	België
▶*	NULL	NULL	NULL	NULL

- Na:

	id	groepnaam	genre	land
▶	126	Clouseau	Pop	NULL
	129	K3	Pop	België
	130	The Bangle	Rock	USA
	131	The Black K...	Rock	USA
	132	Balthazar	Rockabilly	België
	133	De Kreuners	Rockabilly	België
	134	The Scabs	Rockabilly	België
	135	Das Pop	Britpop	België
	136	Soulsister	Pop	België
*	NULL	NULL	NULL	NULL

Achteraf toevoegen van ...

- Toevoegen van velden, constraints aan een bestaande tabel

```
ALTER TABLE Les.Persoon  
  ADD geboortedatum date;
```

```
ALTER TABLE Les.Order  
  ADD CONSTRAINT FK_Order_Klant  
    FOREIGN KEY (klantId)  
    REFERENCES Les.Klant(id);
```

```
ALTER TABLE Les.Persoon  
  ADD email varchar(100);
```

Achteraf verwijderen van ...

- Verwijderen van velden, constraints van een bestaande tabel

```
ALTER TABLE Les.Persoon  
  DROP COLUMN geboortedatum date;
```

```
ALTER TABLE Les.Order  
  DROP CONSTRAINT FK_Order_Klant;
```

Achteraf aanpassen van...

- Velden en constraints aanpassen

```
ALTER TABLE Les.Persoon  
    ALTER COLUMN geboortedatum datetime;
```

Constraints kunnen niet aangepast worden.
Voor constraints:
aanpassen = verwijderen + opnieuw aanmaken

Toevoegen van alternatieve sleutel (AK) of index

- De instructie CREATE INDEX wordt gebruikt om indexen in tabellen te maken.
- Indexen worden gebruikt om snel gegevens uit de database op te halen. De gebruikers kunnen de indexen niet zien, ze worden alleen gebruikt om zoekopdrachten / zoekopdrachten te versnellen.
- Opmerking: Maak alleen indexen op kolommen waarnaar vaak wordt gezocht. Het bijwerken neemt veel tijd in beslag.

Toevoegen van alternatieve key of index

- Syntax:

```
CREATE INDEX AK_Tabelnaam_kolomnaam  
ON Schema.Tabelnaam (kolomnaam1, (kolomnaam2, ...));
```

- Creëert een unieke index op een tabel. Dubbele waarden zijn niet toegestaan.

```
CREATE UNIQUE INDEX AK_Tabelnaam_kolomnaam  
ON Schema.Tabelnaam (kolomnaam1, (kolomnaam2, ...));
```

- Voorbeeld:

```
CREATE UNIQUE INDEX AK_Persoon_email  
ON Les.Persoon (email);
```

Achteraf verwijderen van ...

- Verwijderen van index of AK op een bestaande tabel

```
DROP INDEX Les.Persoon.AK_Persoon_email;
```

Oefeningen

Geldig voor elke nieuwe toepassing of project

Pas volgende stappen steeds toe:

1. Creatie van een schema en creatie van de verschillende tabellen binnen dit schema
2. Toevoegen van de nodige records
3. Uitwerken van de verschillende opdrachten
4. Een aparte SQL-bestand aanmaken om alles terug te verwijderen inclusief constraints, tabellen en schema